Day 6 – HTTP Client & API Integration

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **Backend API Running**: Verify that your Django Orders & Games backend API is running locally (e.g. at `http://127.0.0.1:8000/`).
   - **IDE Ready**: Open `src/app/app.config.ts` and `src/app/services/game.service.ts` in your IDE.
2. **Prerequisites**:
   - You must have completed Day 5 successfully (Reactive forms validation configured and tested locally).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Configure and register Angular's `HttpClient` provider globally.
- Inject `HttpClient` into services and construct HTTP GET and POST requests.
- Translate backend REST JSON payloads to typed TypeScript interfaces.
- Pipe Observable streams and apply RxJS operators (`map`, `catchError`) to handle data transformations.
- Create resilient error-handling pipelines using RxJS `throwError` and catch handlers.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define Client-Server communication. Contrast local memory arrays with remote databases.
2. Key Concepts & Core Ideas	7 min	Explain HttpClient async flows, CORS issues, RxJS piping, and error catch streams.
3. Live Coding Walkthrough	23 min	Enable HttpClient providers, update services to request Django APIs, handle CORS, and manage HTTP failures.

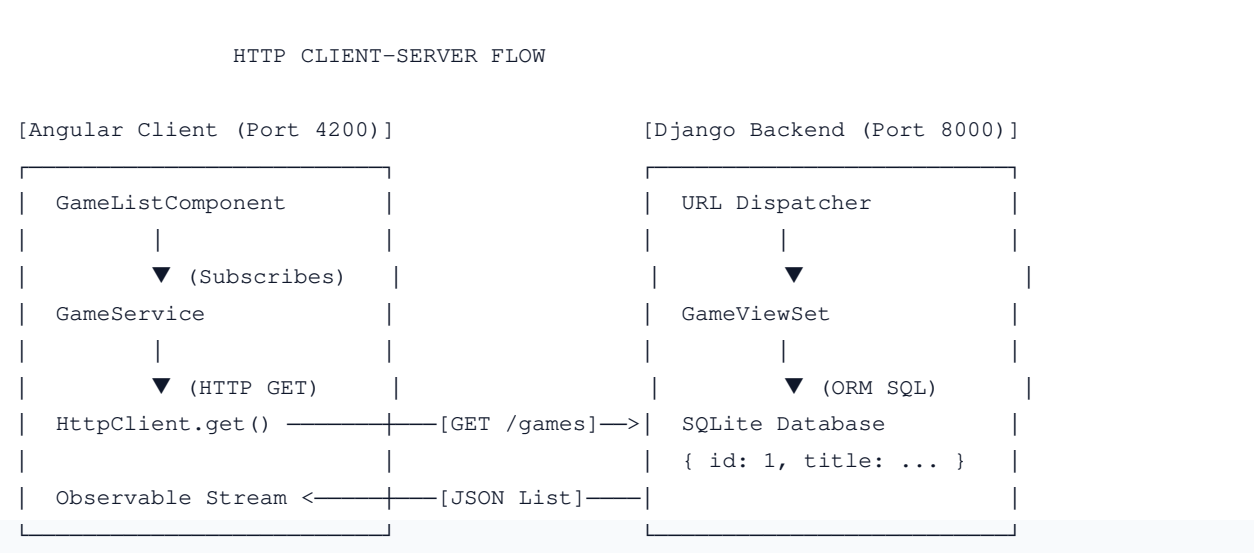
4. Check for Understanding	5 min	Q&A on Observable completion, CORS origin errors, and catching HTTP status codes.
5. Class Wrap-up & Checkpoint	3 min	Verify games catalog fetches data dynamically from the running Django server.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

Up until now, our application has stored game details in a local memory array. Every time you refresh the browser page, any added games disappear. To persist records, we must connect our Angular frontend application to our Django API backend database over the network using HTTP.

- **HTTP Lifecycle**: Component requests data → Service makes asynchronous HTTP call → Angular's `HttpClient` sends request over the web to Django API → Django queries SQLite database and returns JSON payload → Service pipes and maps response → Component receives data and updates the template.



2. Key Concepts & Core Ideas (7 min)

Introduce these integration rules:

- **`HttpClient`**: Angular's built-in service that simplifies making HTTP requests, returning RxJS Observables, parsing JSON response bodies automatically, and intercepting requests/responses.
- **`provideHttpClient()`**: The provider helper function configured in `app.config.ts` to register HTTP client capabilities globally in the DI container.
- **CORS (Cross-Origin Resource Sharing)**: A browser security mechanism that blocks scripts running on

origin `http://localhost:4200` from reading data from another origin `http://127.0.0.1:8000` unless the server sends explicit headers (`Access-Control-Allow-Origin: *`).

- **RxJS `.pipe()`**: A method used on Observables to chain operators together (like filtering, mapping, or handling exceptions) before the data stream reaches the component subscription.
- **`catchError`**: An RxJS operator that catches errors emitted in the Observable stream, allowing you to log the failure and return a fallback value or raise a custom error.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Enabling the HTTP Client Provider

- **Concept**: Configure `app.config.ts` to enable HTTP client services globally.
- **Code**: Modify `src/app/app.config.ts`:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http'; // Import HTTP provider
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient() # Register the HTTP Client provider globally
  ]
};
```

Step 3.2: Updating the Service to Fetch Data via HTTP

- **Concept**: Modify `GameService` to inject `HttpClient` and send GET and POST requests to the Django backend. We will map responses and configure an error-handling pipeline.
- **Code**: Modify `src/app/services/game.service.ts`:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpErrorResponse } from '@angular/common/http'; // Import HttpClient
import { Observable, throwError } from 'rxjs';
import { catchError, map } from 'rxjs/operators'; // Import RxJS pipe operators

export interface Game {
  id: number;
  title: string;
  price: number;
  available?: boolean; // Optional parameter as backend might not contain it
}

@Injectable({
  providedIn: 'root'
```

```
})
export class GameService {
  // URL pointing to the running Django REST API endpoint
  private apiUrl = 'http://127.0.0.1:8000/api/games/';

  // Inject HttpClient into the service constructor
  constructor(private http: HttpClient) {}

  # Fetch games list from the server
  getGames(): Observable<Game[]> {
    // http.get automatically parses JSON responses into typed Game[] arrays
    return this.http.get<Game[]>(this.apiUrl).pipe(
      // catchError intercepts any HTTP failures (e.g. 500 server error)
      catchError(this.handleError)
    );
  }

  # Add a new game record to the database
  addGame(title: string, price: number): Observable<Game> {
    const payload = {
      title: title,
      price: price,
      // Django model requires linking a publisher ID. We pass default publisher ID 1
      publisher: 1
    };

    return this.http.post<Game>(this.apiUrl, payload).pipe(
      catchError(this.handleError)
    );
  }

  # Asynchronous error handler
  private handleError(error: HttpErrorResponse): Observable<never> {
    let errorMessage = 'An unknown error occurred!';
    if (error.error instanceof ErrorEvent) {
      // Client-side or network error
      errorMessage = `Client-side error: ${error.error.message}`;
    } else {
      // Backend returned an unsuccessful response status code
      errorMessage = `Server returned code ${error.status}, body was: ${error.error}`;
    }
    console.error(errorMessage);
    // throwError wraps the message in a new Observable error stream
    return throwError(() => new Error(errorMessage));
  }
}
```

- **Verify**: Check that the service compiles without errors.

Step 3.3: Backend CORS Configuration Check

- **Concept**: If you run the Angular dev server and attempt to load the list, the browser console will log: `Access-Control-Allow-Origin header is missing. CORS origin block`. We must tell our backend

Django server to allow connections from our frontend.

- **Code**: In your Django backend repository:
  1. Add `django-cors-headers` to your Python dependencies (e.g., `uv add django-cors-headers`).
  2. Modify Django settings: Add `'corsheaders'` to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    ...
    'corsheaders',
    ...
]
```

Add `'corsheaders.middleware.CorsMiddleware'` to the top of `MIDDLEWARE`:

```
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # Must be at the top!
    ...
]
```

Add permitted origins configuration to the bottom of the settings file:

```
# Allow requests from the Angular development port
CORS_ALLOWED_ORIGINS = [
    "http://localhost:4200",
    "http://127.0.0.1:4200",
]
```

- **Verify**: Start the Django development server (`python manage.py runserver`). Open `http://localhost:4200/` in your browser. Verify that the catalog page loads the games directly from the SQLite database.

---

4. Check for Understanding (5 min)

> ***Question***: *"Why does Angular's `HttpClient` return Observables instead of standard Javascript Promises?"*
> ***Answer***: *Observables provide a much richer set of features. They can be cancelled mid-flight (e.g. if the user navigates away before a request finishes), they integrate with RxJS operators for transformation and throttling, and they align with Angular's reactive architecture.*

> ***Question***: *"What is CORS, and why is it enforced by the browser?"*
> ***Answer***: *Cross-Origin Resource Sharing (CORS) is a browser-level security feature. It prevents malicious scripts on one website from reading sensitive data from another website's API. A browser*

> *blocks cross-origin requests unless the target server explicitly sends headers authorizing the requesting website's origin.*

> ***Question**: "What does the `catchError` operator return if you want to forward the error to the component?"*
> ***Answer**: `catchError` must return a fallback Observable. To forward the error to the component's subscriber, you return `throwError(() => new Error(message))`, which terminates the stream and triggers the subscriber's `error` callback function.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ The `provideHttpClient` provider is registered in `app.config.ts`.
  - ○ The `GameService` makes real HTTP GET and POST requests.
  - ○ The backend CORS headers are configured, and the frontend successfully renders retrieved items.